APPLIED DEVELOPMENT · JAVA PROJECT

Personal Fitness Tracker

Turning everyday activity into visible progress.

Organized by course outcomes CO1 to CO6, with Java code for each.

[YOUR NAME] · [COURSE CODE]

INTRODUCTION

Personal Fitness Tracker turns daily activity into clear, actionable progress.

A Java console application that records workouts, steps and calories, and calculates BMI.

It connects health habits with practical technology so users can build consistency, not depend on one-time effort.

Advantages and limitations

Advantages

- Activity: one place for workouts, steps, calories and BMI
- Insight: simple summaries show movement toward a goal
- Motivation: visible progress supports repeatable habits

Limitations

- Progress depends on user-entered data
- No integrations, advanced analytics or medical-grade validation
- Summaries support awareness, not medical decisions

DEVELOPMENT CYCLE

Capture, process, visualize, improve.

01

Define

Identify user needs and fitness metrics.

02

Design

Shape the menu flow and data model.

03

Develop

Build tracking logic, storage and feedback.

04

Test

Check validation and calculations.

05

Demonstrate

Show the full user journey.

Technology stack: Java, Scanner, ArrayList, Collections and file I/O.

CO1

From problem to program

Turns a real problem into input, process and output: BMI, BMI category and calories burned.

Problem solving · formulas · input and output · HealthCalculator class

```
double bmi = weightKg / (heightM * heightM);
double cal = met * weightKg * (minutes / 60.0);
```

CO2

Control flow

A switch picks each activity's MET value, if-else gives goal feedback, and loops draw the progress bar and run the menu.

switch · if-else · for · do-while

```
switch (activity.toLowerCase()) {
  case "running": return 9.8;
  case "yoga": return 2.5;
  default: return 4.0;
}
```

CO3

Methods, recursion & arrays

A 7-day step array with reusable methods: recursive sum and max, bubble sort, average and goal-day count.

Methods · recursion · arrays · StepAnalyzer class

```
static int sumRecursive(int[] a, int n) {
  if (n == 0) return 0;
  return a[n - 1] + sumRecursive(a, n - 1);
}
```

CO4

Objects that fail safely

Workout objects validate themselves and throw a custom exception, so bad input never crashes the app.

Encapsulation · custom exception · try/catch

```
if (minutes <= 0 || minutes > 600)
  throw new InvalidInputException("Invalid");
try {
  workouts.add(new Workout(type, m, cal, d));
} catch (InvalidInputException e) {
  System.out.println(e.getMessage());
}
```

CO5

Strings, library algorithms & files

Parses and searches text, sorts with Collections, and saves or loads workouts using try-with-resources.

Strings · Collections.sort · file I/O

```
String[] p = line.split(",");
Collections.sort(workouts);
try (PrintWriter pw = new PrintWriter(
    new FileWriter(fileName))) {
  for (Workout w : workouts)
    pw.println(w.toFileString());
}
```

CO6

Collections & Streams

Workouts live in an ArrayList, and streams filter, sort and total them in a few readable lines.

ArrayList · Stream API · filter · map · collect · WorkoutStats class

```
double total = workouts.stream()
  .mapToDouble(Workout::getCalories)
  .sum();
List<Workout> longOnes = workouts.stream()
  .filter(w -> w.getMinutes() >= 60)
  .sorted()
  .collect(Collectors.toList());
```

FUTURE SCOPE AND CONCLUSION

Future versions can become more personalized, connected and predictive.

Connect

Wearables and health platforms can reduce manual entry.

Personalize

Recommendations can reflect trends, goals and routines.

Extend

A graphical interface, nutrition tracking and a database for many users.

Conclusion: the project makes fitness progress visible and manageable, and brings all the course outcomes together in one working Java program.